

CS 3563: Assignment 2 - Query Optimization Analysis

Task 2: Plan Switch Detection

Asit Patel (CS24BTECH11046)
Shiva Chethan (CS24BTECH11057)
Indian Institute of Technology Hyderabad

March 16, 2026

Introduction

This document outlines the plan switches detected for the three assigned queries as we change the date parameter. A binary search script was used to find the exact adjacent dates where the PostgreSQL query optimizer decided to change its physical execution strategy (the operator tree). Below are the details for each switch, including the dates and a summary of the plan structure before and after the switch.

Query 1

Plan Switch 1: 1992-01-10 to 1992-01-11

- **Plan Before (1992-01-10):** The database executes the query serially using a single process.
 - Aggregate → Bitmap Heap Scan (lineitem) → Bitmap Index Scan
- **Plan After (1992-01-11):** The database switches to parallel execution because the amount of data to process has grown. It adds a **Gather** node to spin up parallel workers.
 - Aggregate → Gather → Aggregate (Partial) → Bitmap Heap Scan (lineitem) → Bitmap Index Scan

Plan Switch 2: 1992-08-21 to 1992-08-22

- **Plan Before (1992-08-21):** The database is still using an index to find rows.
 - Gather → Aggregate → Bitmap Heap Scan (lineitem) → Bitmap Index Scan
- **Plan After (1992-08-22):** The database realizes the date range covers too much of the table, so using an index is no longer worth it. It drops the Bitmap Index Scan entirely and switches to reading the whole table sequentially.
 - Gather → Aggregate → Seq Scan (lineitem)

Query 2

Plan Switch 1: 1993-03-11 to 1993-03-12

- **Plan Before (1993-03-11):** The optimizer uses a Bitmap Index on the `orders` table to find matching dates before joining.
 - Gather Merge → Sort → Nested Loop → Bitmap Heap Scan (`orders`) & Index Scan (`lineitem`)
- **Plan After (1993-03-12):** The date range increases enough that the optimizer drops the Bitmap scan on the `orders` table and switches to a standard Sequential Scan instead.
 - Gather Merge → Sort → Nested Loop → Seq Scan (`orders`) & Index Scan (`lineitem`)

Plan Switch 2: 1993-04-28 to 1993-04-29

- **Plan Before (1993-04-28):** The database uses a `Sorted Aggregate` strategy, which means it sorts the data first, then groups it.
 - Aggregate (Sorted) → Sort → Nested Loop
- **Plan After (1993-04-29):** The database switches its grouping strategy. Instead of a `Sorted Aggregate`, it switches to a `Hashed Aggregate` (`HashAgg`). It does this before the final sort at the top level.
 - Sort → Aggregate (Hashed) → Nested Loop

Plan Switch 3: 1996-11-22 to 1996-11-23

- **Plan Before (1996-11-22):** The database uses a `Nested Loop` to join the tables. For every row in `orders`, it looks up matching rows in `lineitem` using an Index Scan.
 - Nested Loop → Seq Scan (`orders`) & Index Scan (`lineitem`)
- **Plan After (1996-11-23):** The dataset is now huge. Nested Loops are too slow for this many rows, so the optimizer switches to a `Hash Join`. It loads `lineitem` into memory using a Hash and scans `orders` against it sequentially.
 - Hash Join → Seq Scan (`orders`) & Hash (Seq Scan (`lineitem`))

Query 3

Plan Switch 1: 1992-02-28 to 1992-02-29

- **Plan Before (1992-02-28):** The query joins all three tables using Nested Loops.
 - Nested Loop (joins `lineitem`) → Nested Loop (joins `orders` and `customer`)
- **Plan After (1992-02-29):** The database replaces the bottom Nested Loop (between `orders` and `customer`) with a Hash Join.
 - Nested Loop → Hash Join → Bitmap Heap Scan (`orders`) & Hash (Seq Scan (`customer`))

Plan Switch 2: 1993-04-06 to 1993-04-07

- **Plan Before (1993-04-06):** The optimizer uses a Bitmap Heap Scan to fetch data from the `orders` table before hashing it.
 - Hash Join → Bitmap Heap Scan (`orders`) & Hash (`customer`)
- **Plan After (1993-04-07):** The date range increases, so the database drops the index and switches to a simple Seq Scan on the `orders` table.
 - Hash Join → Seq Scan (`orders`) & Hash (`customer`)

Plan Switch 3: 1998-08-28 to 1998-08-29

- **Plan Before (1998-08-28):** The database joins `orders` and `customer` first using a Hash Join, and then joins that result with `lineitem` using an Index Scan.
 - Nested Loop → Hash Join (`orders`, `customer`) & Index Scan (`lineitem`)
- **Plan After (1998-08-29):** The join order completely flips. It now joins `lineitem` and `orders` first using a Nested Loop, and then joins that result with `customer` using a Hash Join.
 - Hash Join → Nested Loop (`lineitem`, `orders`) & Hash (`customer`)

Plan Switch 4: 1998-09-19 to 1998-09-20

- **Plan Before (1998-09-19):** The final top-level join in the tree (joining `customer`) is a Hash Join.
- **Plan After (1998-09-20):** The final top-level join switches to a Nested Loop. The `customer` table is now accessed using an Index Scan inside the loop instead of being hashed.

Plan Switch 5: 1998-11-30 to 1998-12-01

- **Plan Before (1998-11-30):** The `lineitem` table is accessed using a Bitmap Heap Scan. Parallel workers (Gather) are still being used at the top of the tree.
- **Plan After (1998-12-01):** Because the date is very close to the end of the available dataset, very few rows are returned. The optimizer drops the parallel workers entirely and switches the `lineitem` table access to a simple Index Scan.